Name	Martin	James	Jimmy	Ian	Alex
Rules	ERR08-J FIO02-J EXP02-J OBJ10-J EXP00-J	1.SER01-J 2.MET01-J 3.SER04-J 4.SER08-J 5.MET00-J	**SER05-J** **MET06-J** **MET12-J** **MET05-J** **ERR01-J**	**IDS07-J** **IDS16-J** **OBJ05-J** **OBJ11-J** **OBJ13-J**	**1. IDS00-J** **2. FIO08-J** **3. SER12-J** **4. EXP03-J** **5. ERR02-J**

Serialization explained: https://www.baeldung.com/java-serialization

Rule	Priority	Title
IDS00-J	P18	Prevent SQL injection
FIO08-J	P18	Distinguish between characters or bytes read from a stream and -1
SER01-J	P18	Do not deviate from the proper signatures of serialization methods
SER04-J	P18	Do not allow serialization and deserialization to bypass the security manager
SER08-J	P18	Minimize privileges before deserializing from a privileged context
IDS07-J	P12	Sanitize untrusted data passed to the Runtime.exec() method
IDS16-J	P12	Prevent XML Injection
OBJ05-J	P12	Do not return references to private mutable class members
OBJ11-J	P12	Be wary of letting constructors throw exceptions
OBJ13-J	P12	Ensure that references to mutable objects are not exposed
ERR02-J	P12	Prevent exceptions while logging data
ERR08-J	P12	Do not catch NullPointerException or any of its ancestors
FIO02-J	P12	Detect and handle file-related errors

Rule	Priority	Title
SER05-J	P12	Do not serialize instances of inner classes

SEI CERT Oracle Coding Standard for Java — Rules by Priority

Source: CERT Secure Coding Rules index and its Risk Assessment table (Priority = Severity × Likelihood × Remediation Cost).

Excluded groups (per request): 1 – Declarations and Initialization (DCL), 3 – Numeric Types and Operations (NUM), 15 – Platform Security (SEC), 16 – Runtime Environment (ENV), 17 – Java Native Interface (JNI), 49 – Miscellaneous (MSC), 50 – Android (DRD).

Included groups: Input Validation and Data Sanitization (IDS), Expressions (EXP), Characters and Strings (STR), Object Orientation (OBJ), Methods (MET), Exceptional Behavior (ERR), Visibility and Atomicity (VNA), Locking (LCK), Thread APIs (THI), Thread Pools (TPS), Thread-Safety Miscellaneous (TSM), Input Output (FIO), Serialization (SER).

🟥 Highest Priority (P12–P27)

🟧 Mid Priority (P6–P9)

Rule	Priority	Title
EXP02-J	P9	Do not use the Object.equals() method to compare two arrays
EXP03-J	P9	Do not use the equality operators when comparing values of boxed primitives
MET00-J	P9	Validate method arguments
LCK08-J	P9	Ensure actively held locks are released on exceptional conditions
SER12-J	P9	Prevent deserialization of untrusted data
EXP00-J	P8	Do not ignore values returned by methods
OBJ10-J	P8	Do not use public static nonfinal fields
MET01-J	P8	Never use assertions to validate method arguments

Rule	Priority	Title
MET04-J	P8	Do not increase the accessibility of overridden or hidden methods
MET05-J	P8	Ensure that constructors do not call overridable methods
MET06-J	P8	Do not invoke overridable methods in clone()
MET12-J	P8	Do not use finalizers
ERR01-J	P8	Do not allow exceptions to expose sensitive information
VNA00-J	P8	Ensure visibility when accessing shared primitive variables
VNA02-J	P8	Ensure that compound operations on shared variables are atomic
LCK01-J	P8	Do not synchronize on objects that may be reused
LCK02-J	P8	Do not synchronize on the class object returned by getClass()
LCK03-J	P8	Do not synchronize on the intrinsic locks of high-level concurrency objects
LCK06-J	P8	Do not use an instance lock to protect shared static data
TPS04-J	P8	Ensure ThreadLocal variables are reinitialized when using thread pools
TSM01-J	P8	Do not let the this reference escape during object construction
TSM03-J	P8	Do not publish partially initialized objects
IDS01-J	P6	Normalize strings before validating them
IDS11-J	P6	Perform any string modifications before validation
IDS14-J	P6	Do not trust the contents of hidden form fields

Rule	Priority	Title
EXP01-J	P6	Do not use a null in a case where an object is required
OBJ01-J	P6	Limit accessibility of fields
OBJ09-J	P6	Compare classes and not class names
OBJ14-J	P6	Do not use an object that has been freed
MET13-J	P6	Do not assume that reassigning method arguments modifies the calling environment
ERR04-J	P6	Do not complete abruptly from a finally block
ERR07-J	P6	Do not throw RuntimeException, Exception, or Throwable
THI00-J	P6	Do not invoke Thread.run()
FIO05-J	P6	Do not expose buffers or their backing arrays methods to untrusted code
FIO14-J	P6	Perform proper cleanup at program termination
SER00-J	P6	Enable serialization compatibility during class evolution
SER03-J	P6	Do not serialize unencrypted sensitive data
SER06-J	P6	Make defensive copies of private mutable components during deserialization

Lowest Priority (P1–P4)

Rule	Priority	Title
IDS03-J	P4	Do not log unsanitized user input
IDS06-J	P4	Exclude unsanitized user input from format strings
IDS08-J	P4	Sanitize untrusted data included in a regular expression
IDS17-J	P4	Prevent XML External Entity Attacks
EXP04-J	P4	Do not pass arguments to certain Java Collections Framework methods that are a different type than the collection parameter type

Rule	Priority	Title
STR02-J	P4	Specify an appropriate locale when comparing locale-dependent data
OBJ02-J	P4	Preserve dependencies in subclasses when changing superclasses
OBJ03-J	P4	Prevent heap pollution
OBJ06-J	P4	Defensively copy mutable inputs and mutable internal components
OBJ07-J	P4	Sensitive classes must not let themselves be copied
OBJ08-J	P4	Do not expose private members of an outer class from within a nested class
MET03-J	P4	Methods that perform a security check must be declared private or final
MET11-J	P4	Ensure that keys used in comparison operations are immutable
ERR00-J	P4	Do not suppress or ignore checked exceptions
VNA01-J	P4	Ensure visibility of shared references to immutable objects
VNA04-J	P4	Ensure that calls to chained methods are atomic
LCK04-J	P4	Do not synchronize on a collection view if the backing collection is accessible
LCK10-J	P4	Use a correct form of the double-checked locking idiom
THI01-J	P4	Do not invoke ThreadGroup methods
THI05-J	P4	Do not use Thread.stop() to terminate threads
TPS03-J	P4	Ensure that tasks executing in a thread pool do not fail silently

Rule	Priority	Title
TSM00-J	P4	Do not override thread-safe methods with methods that are not thread-safe
TSM02-J	P4	Do not use background threads during class initialization
FIO01-J	P4	Create files with appropriate access permissions
FIO03-J	P4	Remove temporary files before termination
FIO04-J	P4	Release resources when they are no longer needed
FIO07-J	P4	Do not let external processes block on IO buffers
FIO13-J	P4	Do not log sensitive information outside a trust boundary
SER02-J	P4	Sign then seal objects before sending them outside a trust boundary
SER07-J	P4	Do not use the default serialized form for classes with implementation-defined invariants
SER09-J	P4	Do not invoke overridable methods from the readObject() method
EXP06-J	P3	Expressions used in assertions must not produce side effects
OBJ04-J	P3	Provide mutable classes with copy functionality to safely allow passing instances to untrusted code
LCK07-J	P3	Avoid deadlock by requesting and releasing locks in the same order
IDS04-J	P2	Safely extract files from ZipInputStream
EXP05-J	P2	Do not follow a write by a subsequent write or read of the same object within an expression
MET02-J	P2	Do not use deprecated or obsolete classes or methods
MET07-J	P2	Never declare a class method that hides a method declared in a superclass or superinterface

Rule	Priority	Title
MET09-J	P2	Classes that define an equals() method must also define a hashCode() method
MET10-J	P2	Follow the general contract when implementing the compareTo() method
ERR03-J	P2	Restore prior object state on method failure
ERR05-J	P2	Do not let checked exceptions escape from a finally block
VNA03-J	P2	Do not assume that a group of calls to independently atomic methods is atomic
VNA05-J	P2	Ensure atomicity when reading and writing 64-bit values
LCK00-J	P2	Use private final lock objects to synchronize classes that may interact with untrusted code
LCK05-J	P2	Synchronize access to static fields that can be modified by untrusted code
LCK09-J	P2	Do not perform operations that can block while holding a lock
LCK11-J	P2	Avoid client-side locking when using classes that do not commit to their locking strategy
THI02-J	P2	Notify all waiting threads rather than a single thread
THI03-J	P2	Always invoke wait() and await() methods inside a loop
THI04-J	P2	Ensure that threads performing blocking operations can be terminated
TPS00-J	P2	Use thread pools to enable graceful degradation of service during traffic bursts
TPS01-J	P2	Do not execute interdependent tasks in a bounded thread pool
TPS02-J	P2	Ensure that tasks submitted to a thread pool are interruptible

Rule	Priority	Title
FIO00-J	P2	Do not operate on files in shared directories
FIO09-J	P2	Do not rely on the write() method to output integers outside the range 0 to 255
FIO16-J	P2	Canonicalize path names before validating them
SER11-J	P2	Prevent overwriting of externalizable objects
STR00-J	P1	Don't form strings containing partial characters from variable-width encodings
STR01-J	P1	Do not assume that a Java char fully represents a Unicode code point
STR03-J	P1	Do not encode noncharacter data as a string
STR04-J	P1	Use compatible character encodings when communicating string data between JVMs
MET08-J	P1	Preserve the equality contract when overriding the equals() method
ERR06-J	P1	Do not throw undeclared checked exceptions
ERR09-J	P1	Do not allow untrusted code to terminate the JVM
FIO06-J	P1	Do not create multiple buffered wrappers on a single byte or character stream
FIO10-J	P1	Ensure the array is filled when using read() to fill an array
FIO12-J	P1	Provide methods to read and write little-endian data
SER10-J	P1	Avoid memory and resource leaks during serialization

---

Counts: 14 highest-priority, 37 mid-priority, 68 lowest-priority = 119 rules across the 13 included rule groups.